

ORM with Constraints

Pets, owners, and rules, written as Peewee models

Gregory S. DeLozier, PhD

Same Routes, Three Layers

Layer	Job
Browser	Fast feedback
Web application	Reliable movement of data
Database layer	Data follows the rules : Python checks, then the table

Handle the error at every layer we can.

Start with a Current Database

```
python3 upgrade_database.py
```

The file	Result
Missing	Empty current tables
Has the rules the check looks for	No change
Older layout	New tables, data reloaded, original kept

Running the Example

```
python3 -m pip install -r requirements.txt  
python3 upgrade_database.py  
flask run  
python3 -m unittest -v
```

<http://127.0.0.1:5000/list>

Create an owner first.

Rules in the Model

```
class Pet(BaseModel):
    name = TextField(null=False, constraints=[
        Check("length(trim(name)) > 0")])
    type = TextField(null=False, constraints=[
        Check("length(trim(type)) > 0")])
    age = IntegerField(default=0, constraints=[
        Check("age >= 0")])
    owner = ForeignKeyField(Owner, backref="pets",
        null=False, on_delete="RESTRICT")
```

The Table Peewee Writes

```
CREATE TABLE "pet" (  
    "id" INTEGER NOT NULL PRIMARY KEY,  
    "name" TEXT NOT NULL CHECK (length(trim(name)) > 0),  
    "type" TEXT NOT NULL CHECK (length(trim(type)) > 0),  
    "age" INTEGER NOT NULL CHECK (age >= 0),  
    "owner_id" INTEGER NOT NULL,  
    FOREIGN KEY ("owner_id")  
        REFERENCES "owner" ("id") ON DELETE RESTRICT  
)
```

Read it with `sqlite3 pets.db .schema`.

What the Upgrade Reports

```
loaded 1 owners and 2 pets  
2 pets had no owner: now 'Unassigned'  
not carried over: food  
skipped pet 4: CHECK constraint failed: age >= 0  
original file kept as pets.db.before-upgrade
```

Rows that break a rule are **listed, not hidden**.

A Foreign Key Is a Field

Argument	Effect
Owner	Which table
null=False	Every pet has an owner
on_delete="RESTRICT"	Owner with pets cannot go
backref="pets"	owner.pets

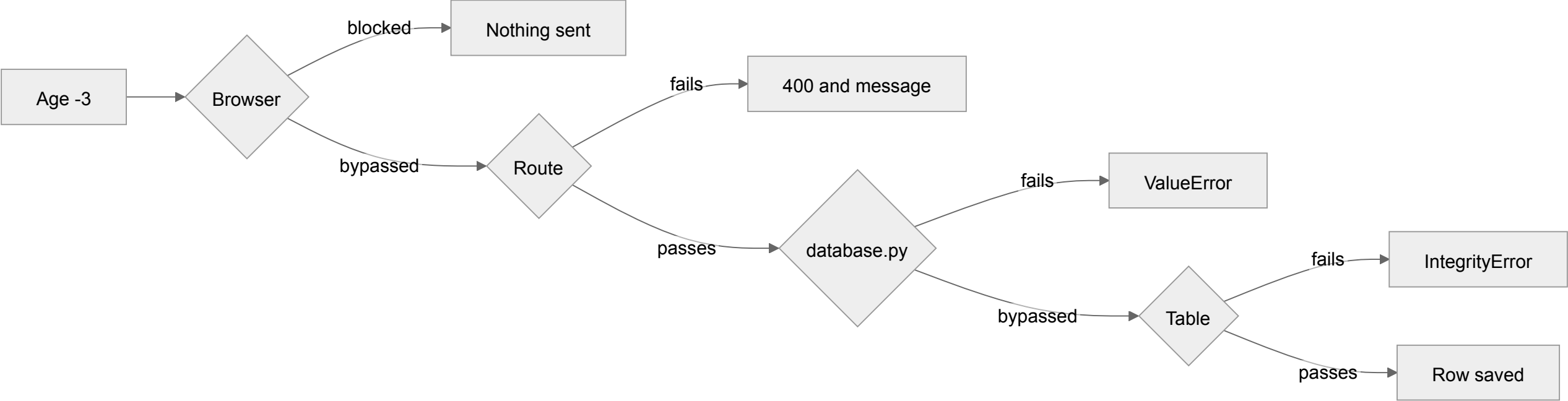
`pet.owner_id` is an integer. `pet.owner` is an `Owner`.

Foreign Keys Are Off Until Asked

```
db.init(database_file,  
        pragmas={"foreign_keys": 1})  
  
...  
fk = db.execute_sql(  
    "PRAGMA foreign_keys").fetchone()[0]  
assert fk == 1
```

The setting belongs to the connection.

One Bad Request, Four Checks



Who Catches What

Request	Browser	Route	database.py	Table
Blank name	required	yes	ValueError	CHECK
Age -1	min="0"	yes	ValueError	CHECK
Age abc	number	yes	ValueError	not stopped
No owner	required	yes	ValueError	NOT NULL
Owner 999999	dropdown	passes it on	passes it on	FOREIGN KEY
Delete owner with pets	none	reports	passes it on	RESTRICT

A Rule Changed

```
age = int(value)
if age < 0:
    raise ValueError("Age must be non-negative.")
return age
```

Blank age is still 0. Bad input is no longer turned into 0.

Errors Arrive as Peewee Exceptions

```
from peewee import IntegrityError, OperationalError
```

```
except IntegrityError as e:  
    return error_page("Error: Cannot delete this "  
        "owner because they have pets.", 400)
```

Routes now depend on Peewee. Is that acceptable?

Join, or Pay N+1

```
Pet.select(Pet, Owner).join(Owner) \
    .order_by(Pet.name, Pet.id)
```

Three pets	Queries
No join, pet.owner.name	4
With join	1

Reassign, Then Delete

```
pet.owner = int(owner_id)
pet.save()
```

Step	Result
Delete Alex, pets remain	Rejected
Move one pet to Sam	Alex still has one
Move the other	Alex has none
Delete Alex	Succeeds

Group Writes Around a Failure

```
with database.db.atomic():  
    database.create_pet(first_pet)  
    database.create_pet(pet_with_bad_owner)
```

Second insert fails: **neither pet is stored.**

Test Below the Layer Above

```
with self.assertRaises(IntegrityError):  
    Pet.create(owner=owner_id, name="    ",  
              type="dog", age=1)
```

Skip the helper. Does the table still refuse?

Exercise: When Is an Owner a Duplicate?

Same name, different cities?

Same name, same city?

Same name, no city?

Which layers should know?

Where the Rules Live

Browser: **tell them now.**

Route: **explain the failure.**

Database layer: **never store a broken row.**